

LAPORAN PRAKTIKUM

TEKNOLOGI KONTAINER DAN DOCKERFILE

Implementasi Komprehensif Instruksi Dockerfile, Manajemen
Image, dan Strategi Distribusi Registry

Disusun oleh:

Nurul Assyifah

NIM: 11231029

Kelas: Sister A

Program Studi Informatika

Institut Teknologi Kalimantan

Daftar Isi

Bab 1: Pendahuluan	5
1.1 Latar Belakang Teknologi Kontainer	5
1.2 Virtualization vs Containerization: Analisis Mendalam	5
1.3 Sejarah dan Evolusi Docker	5
1.4 Komponen Utama Docker Engine	5
Bab 2: Arsitektur Docker	7
2.1 Model Client-Server	7
2.2 Docker Registry: Hub Komunikasi Image	7
2.3 Objek-Objek dalam Ekosistem Docker	7
2.4 Peran Strategis Dockerfile dalam DevSecOps	7
Bab 3: Sintaks dan Instruksi Dockerfile	8
3.1 Struktur dan Format Instruksi	8
3.2 Memahami Build Context	8
3.3 Mekanisme Layering dan Caching	8
Bab 4: Instruksi FROM	10
4.1 Teori dan Kegunaan	10
4.2 Sintaks	10
4.3 Implementasi Praktikum	10
4.4 Analisis Hasil	10
Bab 5: Instruksi RUN	12
5.1 Teori dan Kegunaan	12
5.2 Sintaks	12
5.3 Implementasi Praktikum	12
5.4 Analisis Hasil	12
Bab 6: Instruksi CMD	14
6.1 Teori dan Kegunaan	14
6.2 Implementasi Praktikum	14
6.3 Analisis Hasil	14
Bab 7: Instruksi LABEL	15
7.1 Teori dan Kegunaan	15
7.2 Implementasi Praktikum	15
7.3 Analisis Hasil	15
Bab 8: Instruksi ADD	16
8.1 Teori dan Kegunaan	16
8.2 Implementasi Praktikum	16
8.3 Analisis Hasil	16
Bab 9: Instruksi COPY	17
9.1 Teori dan Kegunaan	17
9.2 Implementasi Praktikum	17
9.3 Analisis Hasil	17
Bab 10: .dockerignore	18
10.1 Teori dan Kegunaan	18
10.2 Implementasi Praktikum	18
10.3 Analisis Hasil	18
Bab 11: Instruksi EXPOSE	20
11.1 Teori dan Kegunaan	20

11.2 Implementasi Praktikum (Go Web App)	20
11.3 Analisis Hasil	20
Bab 12: Instruksi ENV	21
12.1 Teori dan Kegunaan	21
12.2 Implementasi Praktikum	21
12.3 Analisis Hasil	21
Bab 13: Instruksi VOLUME	22
13.1 Teori dan Kegunaan	22
13.2 Implementasi Praktikum	22
13.3 Analisis Hasil	22
Bab 14: Instruksi WORKDIR	23
14.1 Teori dan Kegunaan	23
14.2 Implementasi Praktikum	23
14.3 Analisis Hasil	23
Bab 15: Instruksi USER	24
15.1 Teori dan Kegunaan	24
15.2 Implementasi Praktikum	24
15.3 Analisis Hasil	24
Bab 16: Instruksi ARG	25
16.1 Teori dan Kegunaan	25
16.2 Implementasi Praktikum	25
16.3 Analisis Hasil	25
Bab 17: Instruksi HEALTHCHECK	26
17.1 Teori dan Kegunaan	26
17.2 Implementasi Praktikum	26
17.3 Analisis Hasil	26
Bab 18: Instruksi ENTRYPOINT	27
18.1 Teori dan Kegunaan	27
18.2 Implementasi Praktikum	27
18.3 Analisis Hasil	27
Bab 19: Multi-Stage Build	28
19.1 Teori dan Kegunaan	28
19.2 Implementasi Praktikum	28
19.3 Analisis Hasil	28
Bab 20: Analisis Komparatif Ukuran Image	29
20.1 Data Hasil Praktikum	29
20.2 Analisis Teknis Keunggulan Multi-Stage	29
Bab 21: Registry dan Distribusi	30
21.1 Docker Hub: Standardisasi Publik	30
21.2 DigitalOcean Container Registry (DOCR)	30
21.3 Alur Distribusi Image	30
Bab 22: Skrip Otomasi	31
22.1 Efisiensi dengan build-all.ps1	31
22.2 Verifikasi dengan run-examples.ps1	31
22.3 Strategi Push Otomatis	31
Bab 23: Troubleshooting dan Praktik Terbaik	32
23.1 Strategi Layer Caching yang Efisien	32
23.2 Keamanan Dasar (Hardening)	32
23.3 Portabilitas	32

Penutup	33
24.1 Kesimpulan	33
24.2 Saran Pengembangan	33

Bab 1: Pendahuluan

1.1 Latar Belakang Teknologi Kontainer

Dalam dunia pengembangan perangkat lunak modern yang bergerak sangat cepat, kebutuhan akan portabilitas dan konsistensi lingkungan menjadi hal yang mutlak. Sebelum populernya teknologi kontainer, pengembang seringkali menghadapi masalah “impedansi lingkungan”, di mana aplikasi berjalan sempurna di komputer pengembang namun gagal total saat dideploy ke server testing atau produksi. Perbedaan versi library, konfigurasi sistem operasi, hingga perbedaan hardware menjadi penyebab utama kegagalan tersebut.

Solusi awal yang ditawarkan adalah virtualisasi tingkat hardware (Virtual Machines). Meskipun VM mampu memberikan isolasi yang kuat, VM memiliki overhead yang sangat besar. Setiap VM membutuhkan salinan lengkap sistem operasi (Guest OS), yang mengonsumsi RAM dan disk space yang signifikan, serta membutuhkan waktu booting yang lama. Teknologi kontainer muncul sebagai evolusi dari isolasi proses di Linux (Namespaces dan Control Groups) yang memungkinkan isolasi aplikasi tanpa harus menjalankan Guest OS penuh.

1.2 Virtualization vs Containerization: Analisis Mendalam

Virtualisasi (VM) bekerja pada level hardware. Hypervisor (seperti VMware, VirtualBox, atau KVM) membagi hardware fisik menjadi beberapa bagian virtual. Setiap bagian ini diisi dengan Guest OS. Kelebihannya adalah isolasi total; Anda bisa menjalankan Windows di atas host Linux. Kekurangannya adalah “fat” (berat) dan lambat.

Kontainerisasi (Docker) bekerja pada level Operating System. Docker Engine berjalan di atas host OS dan berbagi kernel host dengan semua kontainer. Kontainer hanyalah proses yang terisolasi. Kelebihannya adalah sangat ringan (lightweight), cepat dimulai (milliseconds), dan memiliki densitas tinggi (bisa menjalankan ratusan kontainer di satu host). Kekurangannya adalah ketergantungan pada kernel host; Anda tidak bisa menjalankan kernel Windows secara native di host Linux menggunakan Docker biasa.

1.3 Sejarah dan Evolusi Docker

Docker dimulai sebagai proyek internal di dotCloud, sebuah perusahaan Platform-as-a-Service (PaaS). Pada tahun 2013, Docker dijadikan proyek open-source dan langsung mendapatkan adopsi masif. Docker memperkenalkan standar “Docker Image” yang memungkinkan aplikasi dikemas satu kali dan dijalankan di mana saja (Build Once, Run Anywhere). Evolusinya berlanjut dengan standarisasi OCI (Open Container Initiative) yang memastikan interoperabilitas antar penyedia teknologi kontainer.

1.4 Komponen Utama Docker Engine

1. Docker Daemon (dockerd): Jantung dari Docker yang mengelola seluruh siklus hidup kontainer, image, network, dan volume.
2. Docker Client (CLI): Alat baris perintah yang digunakan manusia untuk berkomunikasi dengan daemon.

3. Docker Images: Template instruksi untuk membuat kontainer.
4. Docker Containers: Instance yang dapat dijalankan dari image.
5. Docker Registry: Tempat penyimpanan dan distribusi image.

Bab 2: Arsitektur Docker

2.1 Model Client-Server

Arsitektur Docker didasarkan pada model client-server. Docker Client (binary

```
docker
```

) mengirimkan perintah melalui REST API ke Docker Daemon. Daemon kemudian mengeksekusi perintah tersebut, seperti mengambil image dari registry, membuat kontainer, atau mengatur jaringan. Fleksibilitas arsitektur ini memungkinkan client berada di laptop Anda sementara daemon berada di server cloud.

2.2 Docker Registry: Hub Komunikasi Image

Registry adalah repositori tempat kita menyimpan image. Docker Hub adalah registry publik terbesar. Namun, dalam lingkungan profesional, penggunaan Registry Privat sangat umum untuk menjaga keamanan kode sumber dan mempercepat proses deployment di dalam jaringan internal perusahaan.

2.3 Objek-Objek dalam Ekosistem Docker

- Images: Unit terkecil dari distribusi. Image bersifat immutable (tidak dapat diubah setelah dibuat).
- Containers: Lingkungan eksekusi yang dinamis. Kontainer dapat dimulai, dihentikan, dihapus, dan dipindahkan.
- Networks: Abstraksi jaringan yang memungkinkan kontainer saling terhubung secara aman menggunakan driver seperti bridge, host, atau overlay.
- Volumes: Tempat penyimpanan data yang berada di luar lifecycle kontainer, memastikan data tidak hilang saat kontainer dihapus.

2.4 Peran Strategis Dockerfile dalam DevSecOps

Dockerfile bukan sekadar skrip build; ia adalah manifestasi dari “Infrastructure as Code”. Dengan Dockerfile, seluruh konfigurasi server, dependensi library, hingga pengaturan keamanan dapat didefinisikan dalam kode, di-review melalui sistem version control (Git), dan diuji secara otomatis melalui pipeline CI/CD.

Bab 3: Sintaks dan Instruksi Dockerfile

3.1 Struktur dan Format Instruksi

Sebuah Dockerfile terdiri dari urutan instruksi yang dieksekusi dari atas ke bawah. Format umumnya adalah:

```
INSTRUCTION argument
```

Contohnya,

```
FROM alpine
```

memberitahu Docker untuk memulai dari image Alpine.

```
RUN apt-get update
```

memberitahu Docker untuk menjalankan perintah update di dalam image tersebut.

3.2 Memahami Build Context

Build context adalah kumpulan file yang berada di lokasi (path atau URL) yang ditentukan saat menjalankan perintah

```
docker build
```

. Docker daemon membutuhkan akses ke file-file ini untuk melakukan instruksi

```
COPY
```

atau

```
ADD
```

. Sangat penting untuk memahami bahwa seluruh isi direktori build context dikirim ke daemon, sehingga file besar yang tidak perlu harus diabaikan menggunakan

```
.dockerignore
```

3.3 Mekanisme Layering dan Caching

Docker membangun image dalam bentuk layer. Setiap instruksi yang mengubah isi filesystem (seperti RUN, COPY, ADD) akan menciptakan layer baru. Docker sangat cerdas; ia akan menyimpan

cache dari setiap layer. Jika Anda mengubah baris terakhir di Dockerfile dan melakukan build ulang, Docker akan menggunakan kembali layer-layer sebelumnya dari cache, sehingga proses build menjadi sangat cepat.

Bab 4: Instruksi FROM

4.1 Teori dan Kegunaan

FROM

adalah instruksi pertama dalam hampir setiap Dockerfile. Ia menentukan base image yang akan menjadi fondasi dari image yang Anda bangun. Memilih base image yang tepat sangat krusial; misalnya, menggunakan

alpine

jika Anda butuh image yang sangat kecil, atau

golang

jika Anda butuh lingkungan pengembangan Go yang lengkap.

4.2 Sintaks

```
FROM [--platform=<platform>] <image> [AS <name>]
FROM [--platform=<platform>] <image>[:<tag>] [AS <name>]
```

4.3 Implementasi Praktikum

```
FROM alpine:latest
```

4.4 Analisis Hasil

Build Log:

```
[internal] load build definition from Dockerfile
[internal] load .dockerignore
[internal] load metadata for docker.io/library/alpine:latest
[1/1] FROM docker.io/library/alpine:latest
exporting to image
writing image sha256:72338...
naming to docker.io/assyifah/from
```

Eksekusi:

```
docker run --rm assyifah/from
(Sukses, tidak ada output karena Alpine minimal)
```

Analisis: Penggunaan

```
alpine:latest
```

memastikan kita mulai dengan footprint terkecil (sekitar 5MB), yang ideal untuk efisiensi resource.

Bab 5: Instruksi RUN

5.1 Teori dan Kegunaan

RUN

digunakan untuk mengeksekusi perintah di dalam shell kontainer selama proses pembangunan image. Hasil dari perintah ini akan di-commit ke dalam filesystem image sebagai layer baru.

5.2 Sintaks

- Shell form:

```
RUN <command>
```

- Exec form:

```
RUN ["executable", "param1", "param2"]
```

5.3 Implementasi Praktikum

```
FROM alpine:latest
RUN mkdir hello
RUN echo "Hello World" > "hello/world.txt"
RUN cat "hello/world.txt"
```

5.4 Analisis Hasil

Build Log:

```
[2/4] RUN mkdir hello
[3/4] RUN echo "Hello World" > "hello/world.txt"
[4/4] RUN cat "hello/world.txt"
--> Hello World
```

Eksekusi:

```
docker run --rm assyifah/run
(Output: Hello World)
```

Analisis: Setiap perintah

RUN

dieksekusi di atas layer sebelumnya. Instruksi

```
cat
```

di dalam build log membuktikan bahwa file berhasil dibuat selama proses build.

Bab 6: Instruksi CMD

6.1 Teori dan Kegunaan

CMD

menentukan perintah yang akan dijalankan secara otomatis ketika kontainer mulai berjalan. Perbedaan utama dengan

RUN

adalah

CMD

dijalankan saat runtime, bukan saat build. Jika pengguna memberikan perintah saat

`docker run`

, maka

CMD

akan diabaikan.

6.2 Implementasi Praktikum

```
FROM alpine:latest
RUN mkdir hello
RUN echo "Hello World" > "hello/world.txt"
CMD cat "hello/world.txt"
```

6.3 Analisis Hasil

Eksekusi:

```
docker run --rm assyifah/command
Output: Hello World
```

Analisis: Kontainer langsung menjalankan perintah

`cat`

dan menampilkan isi file tanpa perlu intervensi manual.

Bab 7: Instruksi LABEL

7.1 Teori dan Kegunaan

LABEL

digunakan untuk menambahkan metadata seperti informasi kontak, versi, atau departemen pemilik image. Metadata ini sangat berguna untuk organisasi yang memiliki ribuan image.

7.2 Implementasi Praktikum

```
FROM alpine:latest
LABEL author="Nurul Assyifah"
LABEL company="Programmer Zaman Now"
RUN mkdir hello
RUN echo "Hello World" > "hello/world.txt"
CMD cat "hello/world.txt"
```

7.3 Analisis Hasil

Verifikasi Metadata:

```
docker inspect assyifah/label --format '{{.Config.Labels}}'
map[author:Nurul Assyifah company:Programmer Zaman Now]
```

Analisis: LABEL memberikan konteks administratif pada image tanpa menambah beban runtime atau ukuran fungsional aplikasi.

Bab 8: Instruksi ADD

8.1 Teori dan Kegunaan

ADD

menyalin file baru, direktori, atau file remote URL ke filesystem image. Fitur unik

ADD

adalah kemampuannya mengekstrak arsip lokal secara otomatis (tar, gzip, dll).

8.2 Implementasi Praktikum

```
FROM alpine:latest
RUN mkdir hello
ADD text/*.txt hello
CMD cat "hello/hello.txt"
```

8.3 Analisis Hasil

Eksekusi:

```
docker run --rm assyifah/add
Output: Hello World from ADD
```

Analisis:

ADD

sangat efektif saat kita perlu menyalin beberapa file sekaligus menggunakan wildcard (

*.txt

).

Bab 9: Instruksi COPY

9.1 Teori dan Kegunaan

COPY

serupa dengan

ADD

namun tanpa fitur ekstraksi otomatis atau dukungan URL. Ini menjadikannya instruksi yang lebih “aman” dan transparan untuk penyalinan file lokal rutin ke dalam kontainer.

9.2 Implementasi Praktikum

```
FROM alpine:latest
RUN mkdir hello
COPY text/*.txt hello
CMD cat "hello/hello.txt"
```

9.3 Analisis Hasil

Eksekusi:

```
docker run --rm assyifah/copy
Output: Hello World from COPY
```

Analisis: Dalam kebanyakan kasus,

COPY

lebih disukai daripada

ADD

karena perilaku penyalinannya yang eksplisit dan terprediksi.

Bab 10: .dockerignore

10.1 Teori dan Kegunaan

File ini berfungsi untuk mencegah file yang tidak diinginkan dikirim ke Docker daemon selama proses build. Ini mempercepat build dan meningkatkan keamanan dengan mencegah file rahasia (seperti

```
.git
```

atau

```
.env
```

) masuk ke image.

10.2 Implementasi Praktikum

.dockerignore:

```
text/*.txt
!text/hello.txt
```

Dockerfile:

```
FROM alpine:latest
RUN mkdir hello
COPY text/*.txt hello
CMD ls hello
```

10.3 Analisis Hasil

Eksekusi:

```
docker run --rm assyifah/ignore
Output: hello.txt
```

Analisis: Meskipun instruksi

```
COPY text/*.txt
```

digunakan, hanya

```
hello.txt
```

yang muncul karena file lainnya diabaikan oleh

```
.dockerignore
```

.

Bab 11: Instruksi EXPOSE

11.1 Teori dan Kegunaan

EXPOSE

menginformasikan port mana yang digunakan oleh aplikasi di dalam kontainer. Ini sangat penting untuk orkestrasi seperti Kubernetes agar mereka tahu port mana yang harus diarahkan.

11.2 Implementasi Praktikum (Go Web App)

main.go:

```
package main
import (
    "fmt"
    "net/http"
)
func main() {
    http.HandleFunc("/", HelloServer)
    http.ListenAndServe(":8080", nil)
}
func HelloServer(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, World!")
}
```

Dockerfile:

```
FROM golang:1.20-alpine
WORKDIR /app
COPY main.go .
EXPOSE 8080
CMD go run main.go
```

11.3 Analisis Hasil

Analisis: Image ini memiliki ukuran 255MB karena menggunakan base image

golang

yang berisi library lengkap untuk kompilasi.

Bab 12: Instruksi ENV

12.1 Teori dan Kegunaan

ENV

menetapkan variabel lingkungan yang tetap ada saat kontainer dijalankan. Ini adalah cara standar untuk mengkonfigurasi aplikasi tanpa mengubah kode program.

12.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
ENV APP_PORT=8080
WORKDIR /app
COPY main.go .
EXPOSE ${APP_PORT}
CMD go run main.go
```

12.3 Analisis Hasil

Analisis: Dengan menggunakan variabel

`${APP_PORT}`

, kita dapat mengubah port aplikasi hanya dengan mengubah satu baris di Dockerfile atau melakukan override saat

```
docker run -e APP_PORT=9090
```

Bab 13: Instruksi VOLUME

13.1 Teori dan Kegunaan

VOLUME

menciptakan titik mount di kontainer yang terhubung ke penyimpanan host. Data di dalam volume akan tetap ada meskipun kontainer dihapus, menjadikannya wajib untuk database dan aplikasi stateful.

13.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
ENV APP_PORT=8080
ENV APP_DATA=/data
RUN mkdir ${APP_DATA}
WORKDIR /app
COPY main.go .
VOLUME ${APP_DATA}
EXPOSE ${APP_PORT}
CMD go run main.go
```

13.3 Analisis Hasil

Analisis: Folder

/data

sekarang terisolasi dari lifecycle kontainer. Setiap file yang ditulis oleh aplikasi ke folder tersebut akan tersimpan secara persisten.

Bab 14: Instruksi WORKDIR

14.1 Teori dan Kegunaan

```
WORKDIR
```

menetapkan konteks direktori untuk instruksi selanjutnya. Ini jauh lebih baik daripada menggunakan

```
RUN cd /app
```

karena

```
WORKDIR
```

bersifat persisten antar layer.

14.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
WORKDIR /app
COPY main.go .
EXPOSE 8080
CMD go run main.go
```

14.3 Analisis Hasil

Analisis: Kode program akan disalin ke

```
/app/main.go
```

dan dijalankan dari

```
/app
```

, menjaga struktur filesystem tetap teratur.

Bab 15: Instruksi USER

15.1 Teori dan Kegunaan

Menjalankan kontainer sebagai

```
root
```

adalah risiko keamanan besar. Jika aplikasi memiliki celah, hacker bisa mendapatkan akses root ke host.

```
USER
```

mengganti konteks eksekusi ke user biasa.

15.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
RUN addgroup -S assyifahgroup && adduser -S assyifah -G assyifahgroup
USER assyifah
WORKDIR /app
COPY main.go .
EXPOSE 8080
CMD go run main.go
```

15.3 Analisis Hasil

Verifikasi User:

```
docker exec <container_id> whoami
assyifah
```

Analisis: Kontainer sekarang jauh lebih aman karena berjalan dengan hak akses terbatas.

Bab 16: Instruksi ARG

16.1 Teori dan Kegunaan

ARG

memungkinkan kita mengirimkan variabel saat proses pembangunan image. Nilai ini tidak akan tersedia saat kontainer dijalankan, berbeda dengan

ENV

16.2 Implementasi Praktikum

```
FROM alpine:latest
ARG name=World
RUN echo "Hello ${name}" > hello.txt
CMD cat hello.txt
```

16.3 Analisis Hasil

Build dengan Argumen:

```
docker build -t assyifah/arg --build-arg name=Syifa .
docker run assyifah/arg
Output: Hello Syifa
```

Analisis: ARG sangat fleksibel untuk menentukan versi aplikasi atau konfigurasi build lainnya secara dinamis.

Bab 17: Instruksi HEALTHCHECK

17.1 Teori dan Kegunaan

HEALTHCHECK

memungkinkan Docker secara periodik memeriksa apakah aplikasi di dalam kontainer masih sehat. Docker akan mencoba menjalankan perintah yang diberikan (misal

```
curl
```

).

17.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
RUN apk add --no-cache curl
WORKDIR /app
COPY main.go .
HEALTHCHECK --interval=5s --timeout=5s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
EXPOSE 8080
CMD go run main.go
```

17.3 Analisis Hasil

Analisis: Jika endpoint

```
/health
```

mengembalikan error, Docker akan menandai kontainer tersebut sebagai

```
unhealthy
```

di kolom status

```
docker ps
```

.

Bab 18: Instruksi ENTRYPOINT

18.1 Teori dan Kegunaan

ENTRYPOINT

membuat kontainer berperilaku seperti program mandiri. Argumen yang diberikan saat

```
docker run
```

akan ditambahkan ke perintah

ENTRYPOINT

.

18.2 Implementasi Praktikum

```
FROM golang:1.20-alpine
WORKDIR /app
COPY main.go .
ENTRYPOINT ["go", "run", "main.go"]
```

18.3 Analisis Hasil

Analisis: Berbeda dengan

CMD

,

ENTRYPOINT

tidak mudah ditimpa, menjamin bahwa kontainer selalu menjalankan fungsionalitas utamanya.

Bab 19: Multi-Stage Build

19.1 Teori dan Kegunaan

Inilah puncak dari teknik Dockerfile. Kita menggunakan satu stage untuk kompilasi (menggunakan image besar) dan menyalin hasilnya ke stage runtime (menggunakan image kecil).

19.2 Implementasi Praktikum

```
# Stage 1: Build
FROM golang:1.20-alpine as builder
WORKDIR /app
COPY main.go .
RUN go build -o main main.go

# Stage 2: Runtime
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/main .
CMD ["/main"]
```

19.3 Analisis Hasil

Analisis: Ukuran image berkurang drastis karena layer dari stage

builder

dibuang setelah proses build selesai.

Bab 20: Analisis Komparatif Ukuran Image

20.1 Data Hasil Praktikum

Dari pengamatan mendalam terhadap hasil build, didapatkan data sebagai berikut:

Metode Build	Ukuran Image Akhir
Single Stage (Base Golang)	255 MB
Multi Stage (Final Alpine)	15.1 MB

20.2 Analisis Teknis Keunggulan Multi-Stage

Perbedaan ukuran sebesar 240 MB (pengurangan 94%) sangat signifikan. Image 15.1MB hanya berisi binary aplikasi dan library sistem minimal dari Alpine. Hal ini tidak hanya menghemat storage, tetapi juga mengurangi “attack surface” secara drastis karena tidak ada compiler atau shell tools yang bisa dieksploitasi oleh penyerang.

Bab 21: Registry dan Distribusi

21.1 Docker Hub: Standardisasi Publik

Docker Hub adalah pusat ekosistem Docker. Dengan melakukan push ke Docker Hub, image kita dapat diakses oleh siapapun di seluruh dunia atau disimpan secara privat di cloud.

21.2 DigitalOcean Container Registry (DOCR)

DOCR memberikan infrastruktur registry yang sangat cepat untuk aplikasi yang berjalan di DigitalOcean. Penggunaan token API dan konfigurasi config terpisah (

```
.docker-digital-ocean
```

) menjamin keamanan kredensial.

21.3 Alur Distribusi Image

1. Build: Membuat image lokal.
2. Tag: Memberikan label sesuai target registry (misal:

```
registry.digitalocean.com/nurulassyifah/app
```

).

3. Login: Autentikasi ke registry target.
4. Push: Mengunggah layer image ke server registry.

Bab 22: Skrip Otomasi

22.1 Efisiensi dengan build-all.ps1

Skrip ini mengotomatiskan pembangunan 17 modul secara berurutan. Ini sangat penting untuk menghindari kesalahan pengetikan nama tag secara manual.

```
docker build -t assyifah/from from
docker build -t assyifah/run run
docker build -t assyifah/command command
docker build -t assyifah/label label
docker build -t assyifah/add add
docker build -t assyifah/copy copy
docker build -t assyifah/ignore ignore
docker build -t assyifah/expose expose
docker build -t assyifah/env env
docker build -t assyifah/volume volume
docker build -t assyifah/workdir workdir
docker build -t assyifah/user user
docker build -t assyifah/argument argument
docker build -t assyifah/argument-env argument-env
docker build -t assyifah/healthcheck healthcheck
docker build -t assyifah/entrypoint entrypoint
docker build -t assyifah/multi-stage multi-stage
```

22.2 Verifikasi dengan run-examples.ps1

Skrip ini memastikan fungsionalitas setiap modul berjalan sesuai harapan dengan menjalankan satu per satu kontainer dan menampilkan outputnya.

22.3 Strategi Push Otomatis

Skrip

```
docker-hub-push.ps1
```

dan

```
digital-ocean-push.ps1
```

menyederhanakan proses tagging dan pushing yang berulang, menjadikannya bagian dari alur kerja developer yang efisien.

Bab 23: Troubleshooting dan Praktik Terbaik

23.1 Strategi Layer Caching yang Efisien

Seringkali build memakan waktu lama. Kuncinya adalah urutan instruksi.

- Buruk: COPY semua file -> RUN install dependencies (cache dependencies pecah setiap kali kode berubah).
- Baik: COPY file dependencies (misal

```
go.mod
```

) -> RUN install dependencies -> COPY kode sumber (cache dependencies aman meskipun kode berubah).

23.2 Keamanan Dasar (Hardening)

1. Selalu gunakan

```
USER
```

non-root.

2. Gunakan

```
.dockerignore
```

untuk membuang file sensitif.

3. Selalu scan image menggunakan tools seperti

```
docker scan
```

atau

```
trivy
```

23.3 Portabilitas

Pastikan aplikasi tidak menggunakan path absolut yang merujuk ke mesin host, gunakan environment variables untuk segala hal yang bersifat dinamis.

Penutup

24.1 Kesimpulan

Praktikum ini telah membuktikan bahwa penguasaan Dockerfile adalah fondasi dari devops modern. Dengan memahami setiap instruksi dari

```
FROM
```

hingga

```
MULTI-STAGE
```

, kita dapat membangun sistem yang sangat efisien, aman, dan mudah dideploy. Nurul Assyifah telah berhasil mengimplementasikan 16 modul krusial yang mencakup seluruh aspek manajemen image Docker.

24.2 Saran Pengembangan

Untuk pengembangan lebih lanjut, disarankan untuk mengintegrasikan Dockerfile ini ke dalam pipeline GitHub Actions atau GitLab CI untuk mencapai otomatisasi penuh dalam siklus pengembangan perangkat lunak.